

Module 4

AUTOMATION TESTING

From Manual to Automated Testing

- Manual testing involves human testers executing test cases without the assistance of tools or scripts.
- Automation testing, on the other hand, uses software tools to run tests automatically, compare actual outcomes with expected outcomes, and report the results.

Why Should We Switch From Manual To Automation Testing

- Test automation can overcome a lot of manual testing challenges, especially in Agile projects.
- **Test Reusability-** Automated test cases and test suites can be reused multiple times in different test cycles and test environments.
- **Higher Test Coverage-** Since tests are executed automatically, you have more time to focus on new scenarios and write more automated test cases to verify and validate the Application Under Test (AUT).

- **Earlier Feedback and Bug Detection-**

- Bugs escaping into production are more costly to fix, in terms of time, money, and effort.
- Early feedback is important because catching bugs early is much cheaper and faster than fixing them later in production.
- Automated tests, starting from unit tests on developers' local machines, help catch problems right from the start.
- Integrating these tests into a CI (Continuous Integration) pipeline means every time new code is committed, the whole suite of tests runs automatically—unit, integration, system tests—giving immediate feedback.
- This process helps ensure bugs are found and fixed quickly, reducing the chance of defects making it into production.

• **Going Beyond Manual Testing –**

- Test automation can do what manual testing may hardly achieve, especially for specific tests that require lots of effort and resources like performance testing.
- It can simulate numerous simultaneous requests to the AUT server-side from different platforms and devices, which is nearly impossible manually.
- Automation tools work faster than humans, in terms of parallel execution, with higher accuracy. Machines don't get bored or exhausted like humans, hence, they can avoid human-like mistakes. Consequently, the test results are reliable and fast to receive.

First Attempt: Record and Playback

- The first attempt at test automation aimed to replace manual testing is by recording and replaying user actions.
- One way to achieve is to record the actions of the manual tester and then play them over and over again.
- This is commonly done through UI interaction recordings, or alternatively by capturing network traffic (e.g., HTTP requests).

How Record and Playback Works-

- Tester performs actions; the tool records them.
- Recorded actions are saved and can be replayed automatically.
- Intended to reduce repetitive manual regression testing.

- While it's simple to record the actions that the manual tester does and replay them, it's much trickier to detect bugs in an automated manner.
- **Screenshot comparison-** One approach to automatically detect bugs is to compare the image on the screen to the expected image that was recorded.
- Drawback- There could be differences in screen resolution, differences in date and time that appear on the screen, differences in any data that appear on the screen but is not relevant to the test, etc.
- **HTTP network traffic comparison-** Drawback-some of the data may have legitimate differences.

- Even if those technical drawbacks can be addressed by tools, there's still one big drawback- every legitimate change in the application will be considered as a failure, making it hard to distinguish between false positives and actual failures.
- **Human vs Machine in Test Evaluation-**
 - Humans use judgment and context to interpret changes.
 - Machines treat all differences equally—whether they are bugs or valid changes.

- Consequences of Poorly Designed Record & Playback Tests –
 1. Legitimate changes cause repeated test failures until re-recorded.
 2. Re-recording introduces risk of new errors.
 3. Minor UI changes can cause widespread failures (e.g., typo fix in a button).
 4. Tools often fail to give enough diagnostic information for debugging.

- If the expected results of your tests are too general (as in “exactly the way it was”), instead of reflecting only the specific information that is important to test, then your tests will fail much too often due to legitimate changes, rather than on real bugs. In other words, the ratio between false positives and real failures will be too high, which makes them less reliable! Without clear and concise expected results, you’d probably run into the following problems:
1. For every legitimate change in the application, you will keep getting the same failures for subsequent runs of your test until you’ll re-record or fix the test.
 2. When re-recording a test over and over again, there’s a good chance that you’ll have errors in the scenario you’re recording.

3. Often a small change in the application affects many test scenarios. Even if the effect of this change is very minor to a human being, it causes many automatic tests to fail. For example, fixing a typo in a button's text, or even a removal of a redundant space, can cause many tests to fail if they all look for that button's text and click it as part of their scenario.

4. Investigating the result only according to the difference between the actual result and the expected result may not provide enough information that is needed to understand whether it's a bug or a legitimate change.

Differences Between Manual and Automated Testing

1. Definition

- **Manual Testing:** Testing of software manually without using any automated tools or scripts.
- **Automated Testing:** Testing performed using scripts or tools that run tests automatically.

2. Execution

- **Manual Testing:** Executed by human testers.
- **Automated Testing:** Executed by testing tools or frameworks (e.g., Selenium, JUnit).

3. Speed

- **Manual Testing:** Slower, especially for repetitive or complex tests.
- **Automated Testing:** Much faster; suitable for regression and repetitive testing.

4. Accuracy

- **Manual Testing:** Prone to human error.
- **Automated Testing:** More reliable and consistent, given correct scripts.

5. Cost & Time Efficiency

- **Manual Testing:** Lower upfront cost but more expensive over time for large projects.
- **Automated Testing:** Higher initial cost (tools, scripts), but cost-effective long-term.

6. Reusability

- **Manual Testing:** Test cases need to be executed manually each time.
- **Automated Testing:** Test scripts can be reused multiple times across builds.

7. Human Involvement

- **Manual Testing:** Requires full-time human involvement.
- **Automated Testing:** Minimal human intervention once scripts are ready.

8. Skill Requirement

- **Manual Testing:** Basic software testing knowledge required.
- **Automated Testing:** Requires programming/scripting and tool knowledge.

9. Types of Testing

- **Manual Testing:**

- Exploratory Testing-testers adapt and experiment based on observations, using their skills to find problems quickly.
- Usability Testing-method used to evaluate the user experience and navigation of websites, apps, and digital products.
- Ad hoc Testing-testing method where testers explore an application without a predefined test plan.

- **Automated Testing:**

- Regression Testing-When developers make changes to a program, regression testing checks that the old parts still work as expected after those changes.
- Load & Performance Testing-Used to evaluate how a system behaves under various conditions, especially under stress or heavy usage.

Importance of Software Testing

- Software testing is a crucial phase in the software development lifecycle that ensures a product is reliable, functional, secure, and ready for release.
- Below are the key reasons why software testing is important:

1. Ensures Software Quality

- Confirms that the software performs as expected under various conditions.
- Verifies that all requirements (functional and non-functional) are met.

2. Detects and Fixes Bugs Early

- Early detection of defects reduces the cost and effort of fixing them later.
- Helps maintain the stability and integrity of the product throughout development.

3. Improves Security

- Identifies vulnerabilities that could be exploited by attackers.
- Ensures data protection and compliance with security standards.

4. Enhances User Satisfaction

- A well-tested product is more likely to be user-friendly, efficient, and free of errors.
- Leads to higher user confidence and better customer retention.

5. Supports Cost Efficiency

- Prevents costly post-release bugs and system failures.
- Saves money by minimizing the risk of project rework, recalls, or legal issues.

6. Ensures Compatibility

- Tests the software across different devices, platforms, and browsers.
- Guarantees consistent performance for all users.

7. Validates Business Requirements

- Ensures the software aligns with business goals and user needs.
- Confirms that all use cases and scenarios are correctly implemented.

Relation of Test automation to Software development aspects

1. Architecture

- Well-structured and modular architectures allow easier testing of individual components, while poor design can make creating and maintaining automated tests difficult.

2. Working Practices

- Test automation fits into modern development practices like Agile and DevOps. It supports continuous integration (CI) and continuous delivery (CD), providing rapid feedback to developers and helping to catch bugs early.

3. Quality and Productivity

- Automated testing ensures consistent software quality by detecting defects quickly and reliably. It also saves time and effort by reducing repetitive manual testing, allowing developers and-testers to focus on more critical tasks.

4. Business Structure

- Teams need to view automation as a shared responsibility rather than leaving it to testers alone. This mindset ensures smoother implementation and better results.

5. Documentation

- Good documentation is crucial for test automation. It helps teams understand how the test framework works and how to use or update it. Clear and detailed documentation makes automation easier to maintain and helps new team members get up to speed quickly.